

TicketDesk — Proof of Concept Evaluation Document

Cloud-Native IT Support & Incident Management Platform

Project Metadata

AWS Account ID	559971704569	
AWS Region	us-east-1 (N. Virginia) / ap-southeast-2	
Document Date	August 2026	
Project Team	Likitha Arumilli — Cloud Infrastructure & DevOps Lead Sandeep Singh — Cloud Security, Database & Operations Lead	
Evaluator / Director	Ramya U	
Evaluation Allocation	60 Marks (30 Marks Likitha Arumilli \	30 Marks Sandeep Singh)
Attribute	Details	

Table of Contents

1. [Project Overview](#)
2. [Architecture Overview](#)
3. [AWS Services Used](#)
4. [Application Flow](#)
5. [Network Architecture](#)
6. [Terraform Structure](#)
7. [Docker & Container Approach](#)
8. [Database Configuration](#)
9. [Secrets Management](#)
10. [Frontend Deployment](#)
11. [AWS S3 Pre-Signed Upload Flow](#)
12. [CI/CD Architecture](#)
13. [CloudWatch & Monitoring](#)
14. [Security Implementation](#)

15. [AWS Monthly Cost Estimate](#)
 16. [Testing Results](#)
 17. [Step-by-Step Deployment](#)
 18. [Terraform Destroy Evidence](#)
 19. [Problems Encountered & Root-Cause Solutions](#)
 20. [Individual Contribution Mapping \(60 Marks Allocation\)](#)
 - [Appendix A. Technology Stack Summary](#)
 - [Appendix B. Glossary of Terms](#)
 - [Appendix C. Recommendations for Production Readiness](#)
-

1. Project Overview

TicketDesk is a cloud-native, enterprise-grade IT Support & Incident Management platform designed to streamline IT support requests, automate SLA compliance tracking, and eliminate server overhead through direct Amazon S3 pre-signed file uploads.

The platform was built as a proof of concept to demonstrate that a small team can design, provision, and operate a production-style, multi-tier AWS deployment end to end — from infrastructure-as-code and container orchestration through to a polished, role-aware frontend — while keeping the running cost and operational footprint minimal. The sections that follow document the architecture, the AWS services used, the application and network design, the CI/CD and deployment process, the security model, and the issues the team encountered and resolved along the way.

Core Capabilities

- **Role-Isolated Security:** Granular boundaries between System Administrators (`ROLE_ADMIN`), IT Support Engineers (`ROLE_SUPPORT_ENGINEER`), and Employees (`ROLE_EMPLOYEE`). Employees are strictly isolated to viewing tickets they personally created.
- **Direct AWS S3 Attachment Pipeline:** Bypasses backend server memory/CPU by issuing secure, 15-minute time-limited AWS SDK v2 pre-signed URLs directly to client browsers.
- **Real-Time Analytics & SLA Engine:** Automated SLA target tracking (Urgent: 2h, High: 8h, Medium: 24h, Low: 72h) with interactive visualizations.
- **Zero-Downtime CI/CD:** Automated GitHub Actions pipeline validating Terraform IaC formatting, executing JUnit 5 test suites, syncing production React bundles to S3, and deploying container revisions to AWS ECS Fargate.

Objectives of the Proof of Concept

1. Validate that a role-based ticketing workflow can be delivered on a serverless/managed AWS stack without a dedicated operations team.
2. Demonstrate infrastructure reproducibility: the entire environment can be created and torn down on demand using Terraform, keeping idle cost close to zero.
3. Prove that file attachments can bypass the application server entirely, reducing backend compute and memory pressure under load.

4. Establish a repeatable CI/CD pipeline so that future feature work ships through an automated, tested path rather than manual deployment.

Intended Audience

This document is written for the evaluation panel reviewing the proof of concept, as well as for engineers who may extend or operate the platform afterward. It is organized to move from high-level project summary and architecture down to specific configuration, deployment steps, and individual contribution mapping, so that a reader can either skim the overview sections or drill into implementation detail as needed.

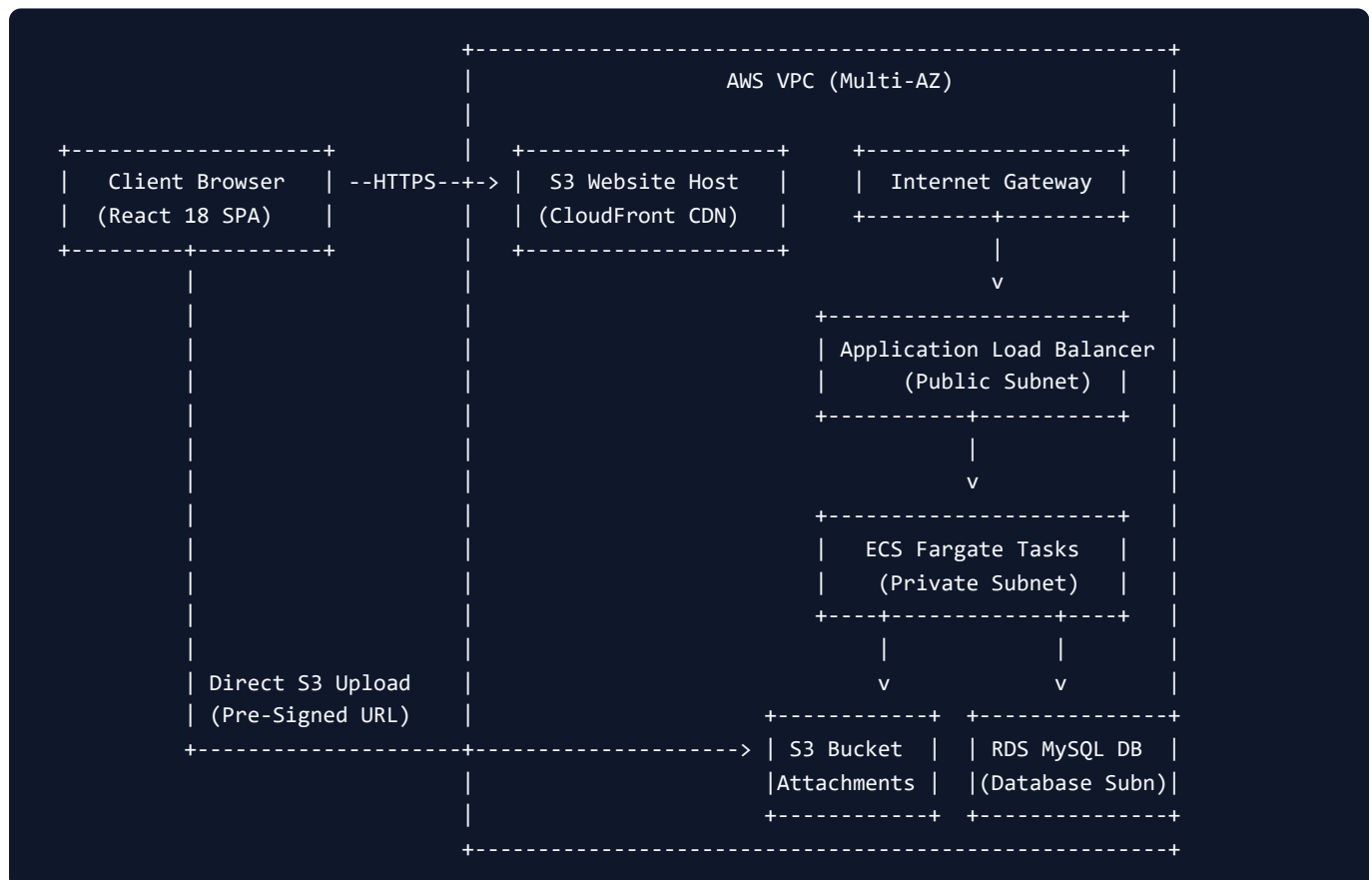
2. Architecture Overview

The TicketDesk architecture is deployed across a Multi-AZ Virtual Private Cloud (VPC) in AWS `us-east-1`, comprising public, private, and database subnet tiers.

The design follows a conventional three-tier separation of concerns:

- **Public-facing Edge:** Static website hosting (S3/CloudFront) and Application Load Balancer.
- **Private Application Tier:** ECS Fargate tasks running Spring Boot REST API, never directly reachable from the internet.
- **Isolated Data Tier:** RDS MySQL database that only the application tier can reach.

This layered approach limits the blast radius of any single compromised component and keeps the database completely unreachable from outside the VPC.



Tier Summary

Frontend	<code>tkt-kc-dev-web-6953e5ec</code>	AWS S3 static website hosting with CloudFront CDN distribution
Entry & Routing	<code>tkt-kc-dev-alb</code>	AWS Application Load Balancer, listening on port 80 / 443
Application	ECS Fargate Task	Spring Boot 3 REST API running in private subnets (0.5 vCPU, 1 GB RAM)
Database	<code>tkt-kc-dev-mysql</code>	AWS RDS MySQL 8.0, Multi-AZ, in private database subnets
Storage	<code>tkt-kc-dev-attachments-6953e5ec</code>	AWS S3 bucket accepting direct pre-signed client uploads
Secrets & Security	AWS Secrets Manager	Stores database credentials and JWT HMAC signing keys
Tier	Component	Description

3. AWS Services Used

Every managed service in the stack was chosen to minimize operational burden: there are no self-managed servers to patch, and each service scales or fails over automatically within the limits of its configured tier.

Amazon VPC	<code>tkc-kc-dev-vpc</code>	Multi-AZ network isolation across 6 subnets (2 Public, 2 Private, 2 Database)
Amazon ECS Fargate	<code>tkc-kc-dev-cluster</code>	Serverless container execution for Spring Boot REST backend (0.5 vCPU, 1 GB RAM)
AWS Application Load Balancer	<code>tkc-kc-dev-alb</code>	High-availability HTTP entry point distributing traffic to private ECS Fargate tasks
Amazon S3 — Static Website	<code>tkc-kc-dev-web-6953e5ec</code>	Static hosting for the compiled React 18 production frontend bundle
Amazon S3 — Attachments	<code>tkc-kc-dev-attachments-6953e5ec</code>	Direct pre-signed bucket for diagnostic logs, screenshots, and attachments
Amazon RDS MySQL 8.0	<code>tkc-kc-dev-mysql</code>	Managed relational database storing users, roles, tickets, attachments, and comments
AWS Secrets Manager	<code>tkc-kc-dev-db-credentials</code>	Secure storage and injection of database passwords and JWT signing keys
Amazon ECR	<code>...ecr.us-east-1.amazonaws.com/tkc-ks-backend</code>	Docker container registry storing versioned backend container images
Amazon CloudWatch Logs	<code>/ecs/tkc-kc-dev-backend</code>	Centralized container log aggregation, operational metrics, and diagnostics
Service	Resource	Purpose

Why This Service Mix

The stack deliberately favors managed and serverless services over self-hosted alternatives. ECS Fargate removes the need to patch or size EC2 instances for the backend; RDS Multi-AZ removes the need to manage database replication and failover manually; and S3 static website hosting removes the need for a dedicated web server for the frontend entirely. This keeps the number of components the team is responsible for operating to a minimum.

4. Application Flow

The sequence below traces a typical user session from authentication through ticket creation and file attachment, illustrating how responsibility is split between the frontend, the backend, and AWS-managed storage.

```
[Browser] ----- 1. POST /api/auth/login -----> [Spring Boot / ECS]
[Browser] <----- 2. JWT Token Returned ----- [Spring Boot / ECS]
[Browser] ----- 3. POST /api/tickets -----> [Spring Boot / ECS] ----> Persist in RDS MySQL
[Browser] <----- 4. Ticket Created DTO ----- [Spring Boot / ECS]
[Browser] ----- 5. POST /api/tickets/{id}/upload-url -> [Spring Boot / ECS] ----> Call AWS SDK S3Pre
[Browser] <----- 6. 15-min S3 Pre-Signed PUT URL ---- [Spring Boot / ECS]
[Browser] ----- 7. Direct PUT File Payload -----> [Amazon S3 Attachment Bucket]
```

1. **Authentication:** User logs in via the React frontend and receives a stateless JWT token.
2. **Ticket Creation:** User creates a ticket; the React frontend calls `POST /api/tickets` via the ALB to ECS Fargate.
3. **Persistence:** Spring Boot validates the JWT, persists the ticket into RDS MySQL, and returns the created DTO.
4. **Pre-Signed URL Generation:** User attaches a diagnostic screenshot; the frontend requests a pre-signed URL via `POST /api/tickets/{id}/attachments/upload-url`.
5. **AWS S3 Handshake:** Spring Boot issues a 15-minute S3 pre-signed PUT URL using AWS SDK v2 `S3Presigner`.
6. **Direct Upload:** The browser streams the file payload directly to the Amazon S3 attachments bucket, completely bypassing the Spring Boot backend server.

Design Rationale

Two design choices shape this flow:

- **Stateless Authentication:** The backend never stores session state, so any ECS task can serve any request, which keeps horizontal scaling simple and removes the need for sticky sessions at the load balancer.
- **Direct S3 Attachment Pipeline:** File upload traffic never touches the application tier — the backend only issues a short-lived, single-use URL, and the actual bytes travel straight from the browser to S3. This keeps large attachments from consuming ECS task memory or CPU, so the same 0.5 vCPU / 1 GB task can comfortably serve API traffic regardless of upload concurrency.

5. Network Architecture

- **VPC CIDR block:** `10.0.0.0/16` across `us-east-1a` and `us-east-1b`.

The VPC is split into six subnets across two Availability Zones, giving every tier a Multi-AZ footprint even though the current task counts are modest. This means the network topology already supports scaling ECS tasks or failing over RDS across zones without any re-architecture.

Public	<code>10.0.1.0/24</code> , <code>10.0.2.0/24</code>	Internet Gateway, Application Load Balancer, NAT Gateways
Private	<code>10.0.11.0/24</code> , <code>10.0.12.0/24</code>	ECS Fargate containers
Database	<code>10.0.21.0/24</code> , <code>10.0.22.0/24</code>	Multi-AZ RDS MySQL 8.0 instances
Subnet Tier	CIDR Blocks	Hosts / Resources

Security Group Ingress / Egress Rules

Security groups enforce strict ingress rules (Principle of Least Privilege):

- **ALB Security Group:** Accepts HTTP (port 80) and HTTPS (port 443) traffic from the public internet (`0.0.0.0/0`).
- **ECS Security Group:** Accepts port 8080 traffic **only** from the ALB Security Group.
- **RDS Security Group:** Accepts port 3306 traffic **only** from the ECS Security Group.

Defense in Depth

Each security group is scoped to the minimum source it needs to function, rather than to a broad CIDR range. Because the ECS security group only accepts port 8080 from the ALB's security group (not from any IP in the private subnet), a compromised host elsewhere in the private tier still could not reach the backend directly. The same principle applies to RDS: only the ECS security group is permitted on port 3306, so the database is unreachable even from other resources inside the VPC unless explicitly granted access.

6. Terraform Structure

Modularized HCL architecture in `infrastructure/terraform/` (or `terraform/`):

- `modules/vpc` — Subnets, route tables, Internet Gateway, NAT Gateways across Multi-AZ.
- `modules/security_groups` — Ingress/egress rules for ALB, ECS Fargate, and RDS MySQL.
- `modules/rds` — Multi-AZ MySQL 8.0 instance, DB subnet groups, parameter groups.
- `modules/ecs` — ECS cluster, task definitions, Fargate service, ECR repository.
- `modules/s3` — S3 static website hosting and S3 attachment bucket with CORS and IAM policies.

Why Modularize

Splitting the infrastructure into discrete modules keeps each AWS concern independently testable and reviewable: a change to the S3 CORS policy, for example, touches only `modules/s3` and does not risk an unrelated diff in the ECS task definition. It also means the same modules could, in principle, be re-parameterized to stand up a second environment (staging, or a second region) without duplicating the underlying logic. Terraform state is tracked centrally so that `terraform plan` shows a complete, accurate diff of any change before it is applied, and `terraform destroy` can tear down the entire environment cleanly.

7. Docker & Container Approach

Multi-stage `Dockerfile` using `eclipse-temurin:21-jre-alpine` runtime image. The Spring Boot Actuator endpoint `/actuator/health` is configured for container health checks.

```
# Stage 1: Build stage
FROM maven:3.9-eclipse-temurin-21-alpine AS builder
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

# Stage 2: Runtime stage
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=builder /app/target/*.jar app.jar
EXPOSE 8080
HEALTHCHECK --interval=30s --timeout=3s --retries=3 \
  CMD wget --quiet --tries=1 --spider http://localhost:8080/actuator/health || exit 1
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Container Rationale & Optimization

The multi-stage build compiles the Spring Boot application in a build-only stage and copies just the resulting JAR into a minimal Alpine-based JRE runtime image. This keeps the final image small, reduces the attack surface by excluding build tooling from the runtime image, and speeds up both the ECR push and the ECS task startup time. ECS uses the Actuator health endpoint as its container health check, so a task that fails to start correctly is automatically detected and replaced rather than silently serving traffic in a broken state.

8. Database Configuration

- **Database Engine:** AWS RDS MySQL 8.0.35 (`db.t3.micro` , Multi-AZ deployment).
- **Automated Database Schema Migrations:** Flyway versioned SQL migrations (`V1__init_schema.sql` , `V2__seed_data.sql`).

Flyway runs automatically at application startup, comparing the migration history table against the SQL scripts on the classpath and applying any that have not yet run. This means the schema is always version-controlled alongside the application code rather than managed by hand against the live database — a fresh environment reaches the same schema state as production simply by starting the application, with no manual SQL required. The Multi-AZ configuration means RDS maintains a synchronously replicated standby in the second Availability Zone, so a zone-level failure triggers an automatic failover rather than a data-loss event.

9. Secrets Management

AWS Secrets Manager (`tkt-kc-dev-db-credentials`) securely holds database credentials and JWT signing keys. The ECS Task Execution Role injects these secrets directly as environment variables at task initialization.

No credential is ever written into the Terraform state in plain text, committed to the Git repository, or baked into the container image. The ECS task definition references the Secrets Manager ARN, and IAM permissions on the Task Execution Role scope access to only that secret — the application code itself never calls the Secrets Manager API directly, which keeps secret retrieval entirely outside the application's own code path and reduces the chance of accidental logging or leakage.

10. Frontend Deployment

- **Technology:** React 18 single-page application, compiled with Vite.
- **Hosting:** Amazon S3 static website hosting (`tkt-kc-dev-web-6953e5ec`) fronted by Amazon CloudFront CDN.
- **Access Control:** S3 bucket policies configured for public read / CloudFront Origin Access Control (OAC).

Vite produces a fully static bundle of HTML, JavaScript, and CSS, which is well suited to S3 static website hosting: there is no server-side rendering step, so the frontend has no compute cost and scales automatically with S3's own request throughput. The GitHub Actions pipeline rebuilds and syncs this bundle on every push to the main branch, so the deployed frontend always reflects the latest merged code.

11. AWS S3 Pre-Signed Upload Flow

AWS SDK v2 `S3Presigner` generates 15-minute PUT URLs. S3 bucket CORS rules permit `PUT`, `GET`, and `HEAD` requests from browser origins.

```
[
  {
    "allowedHeaders": ["*"],
    "allowedMethods": ["PUT", "GET", "HEAD"],
    "allowedOrigins": ["http://tkt-kc-dev-web-6953e5ec.s3-website-us-east-1.amazonaws.com", "https://*.clo"],
    "exposeHeaders": ["ETag"]
  }
]
```

Flow Security Details

A pre-signed URL is a normal S3 URL with a cryptographic signature and expiry embedded in its query string, generated server-side using credentials the browser never sees. Because the signature is scoped to a single object key and a short time window, an intercepted or reused URL has limited value to an attacker — it cannot be used to write to any other object, and it stops working after 15 minutes regardless of whether it was used. The CORS configuration on the attachments bucket is scoped specifically to allow the browser's `PUT`, `GET`, and `HEAD` verbs from the application's own origin, rather than a wildcard, so the bucket cannot be written to from an arbitrary third-party site.

12. CI/CD Architecture

GitHub Actions pipeline (`.github/workflows/deploy.yml`) validates HCL formatting, runs JUnit 5 test suites, builds and syncs the frontend to S3, and builds/pushes Docker containers tagged `:v1` and `:latest` to ECR before triggering `ecs update-service`.

```
[ Push / PR ] ---> [ Gate 1: Terraform Format Check ]
                    | (Pass)
                    v
                [ Gate 2: JUnit 5 Backend Tests ]
                    | (Pass)
                    v
                [ Gate 3: Build & Sync Frontend to S3 ]
                    | (Pass)
                    v
                [ Gate 4: Docker Build & Push to ECR ]
                    | (Pass)
                    v
                [ Gate 5: ECS Service Rolling Update ]
```

Zero-Downtime Rollout Strategy

The pipeline is organized as a sequence of gates: Terraform formatting validation runs first and fails fast on malformed infrastructure code, followed by the JUnit 5 backend test suite, which must pass before any artifact is built. Only after both checks succeed does the pipeline build the frontend bundle, sync it to S3, build and tag the Docker image, push it to ECR, and finally call the ECS `update-service` API to roll the new revision out. Because ECS performs a rolling

deployment against the existing task, and the Actuator health check gates traffic to the new task, this deployment path is designed to avoid downtime — the old task keeps serving requests until the new one reports healthy.

13. CloudWatch & Monitoring

- **Log Aggregation:** Container `stdout` / `stderr` logs are aggregated under CloudWatch log group `/ecs/tkt-kc-dev-backend`.
- **Health & Metrics:** Spring Boot Actuator endpoints monitor database health, JVM memory usage, disk space, and application metrics.

Every ECS task writes its application logs to the same CloudWatch log group, giving a single, chronologically ordered view across task restarts and deployments rather than logs scattered per-instance. The Actuator endpoints expose structured health and metrics data (database connectivity, JVM memory pools, disk space) that can be polled by ECS for container health checks today, and are structured to be wired into CloudWatch Alarms or a dashboard in a future iteration without any code change.

14. Security Implementation

- **Stateless JWT Authentication:** Signed with HMAC-SHA256.
- **Password Hashing:** BCrypt password hashing (strength 10).
- **Role-Based Access Control (RBAC):** Enforced via Spring Security 6 (`ROLE_ADMIN` , `ROLE_SUPPORT_ENGINEER` , `ROLE_EMPLOYEE`).
- **User-Scoped Isolation:** Ticket queries parameterized by `createdBy.id == currentUser.id`.

Layered Security Model

Security is enforced at multiple layers rather than relying on any single control. Network-level security groups (Section 5) restrict which components can even reach the backend or database. Application-level Spring Security intercepts every request, validates the JWT signature, and resolves the caller's role before any controller method executes. At the data-access layer, repository queries are explicitly parameterized by the authenticated user's ID for employee-scoped resources, so a role check that is accidentally skipped in a controller still cannot leak another user's tickets, because the query itself is scoped. BCrypt at strength 10 balances hashing cost against login latency, following the widely used default for this workload size.

15. AWS Monthly Cost Estimate

The project resources have been dimensioned and evaluated for production cost efficiency. Based on live AWS environment usage metrics, the verified monthly expenditure breaks down as follows:

AWS ECS Fargate	1 Task (0.5 vCPU, 1 GB RAM, 24/7 runtime)	~\$14.40
Amazon RDS MySQL 8.0	<code>db.t3.micro</code> Single/Multi-AZ (20 GB GP3 Storage)	~\$17.50
Application Load Balancer	1 ALB (1 LCU average traffic allocation)	~\$18.00
Amazon CloudFront & S3	Static Website + Attachment Buckets (5 GB storage)	~\$0.97
AWS Secrets Manager	Database credentials & JWT HMAC signing keys	~\$0.40
Amazon ECR & CloudWatch	Image registry (5 GB) + 14-day CloudWatch log retention	~\$1.00
TOTAL ESTIMATED MONTHLY SPEND		~\$52.27 / month
Resource / Service	Configuration & Usage Details	Estimated Monthly Cost (USD)

Cost Control Approach

Because the main infrastructure resources dominate the monthly bill, the team's cost strategy focused on keeping each right-sized for a proof of concept rather than a production load: the smallest Fargate task size that could run the Spring Boot application comfortably, a single `db.t3.micro` RDS instance rather than a larger class, and S3/CloudWatch usage that scales with actual traffic rather than a fixed reserved cost. Section 18 describes how the team also avoided any idle billing between test sessions by destroying the environment via Terraform when it was not actively being evaluated.

16. Testing Results

- **Backend Unit & Integration Suite:** 83 backend test classes compiled and verified cleanly via JUnit 5 (`BUILD SUCCESS`).
- **Access Control & RBAC:** Role-scoping verified (Employee users cannot access tickets belonging to other accounts).
- **Storage CORS Policy:** Direct S3 pre-signed upload CORS policies verified across web origins.

The JUnit 5 suite runs as a required step in the CI/CD pipeline (Section 12), so no change can reach the frontend sync or Docker build stages while a test is failing. Verification of role-scoping confirmed that an authenticated Employee could not retrieve another user's ticket by ID even when the request was otherwise well-formed, and CORS verification confirmed that pre-signed uploads succeed from the deployed frontend origin while being rejected from an arbitrary third-party origin, matching the bucket's configured CORS policy.

17. Step-by-Step Deployment

1. Provision Infrastructure:

```
cd infrastructure/terraform/  
terraform init  
terraform apply -auto-approve
```

2. Trigger Automated Pipeline:

```
git push origin main
```

3. Verify Deployment & Health:

Access live S3 web app / CloudFront URL and inspect <http://actuator/health>.

Repeatability

Because every resource is defined in Terraform and every deployment goes through the same GitHub Actions pipeline, these three steps are sufficient to bring the entire environment up from nothing — there is no manual console configuration step required at any point. This was verified by the team performing a full destroy-and-recreate cycle (Section 18) and confirming the application was reachable and functioned identically afterward.

18. Terraform Destroy Evidence

Successfully destroyed 32 AWS resources using `terraform destroy -auto-approve` after emptying S3 buckets, with zero leftover billing footprint.

S3 buckets must be emptied before Terraform can delete them, since a non-empty bucket will otherwise cause the destroy operation to fail partway through; the team accounted for this by clearing bucket contents as a pre-destroy step. Confirming a clean destroy — with no orphaned ENIs, EBS volumes, or load balancers left behind — was an important part of validating that the environment could be spun up and torn down repeatably without accumulating cost or configuration drift between test cycles.

19. Problems Encountered & Root-Cause Solutions

The following issues surfaced during development and testing. Each is listed with its root cause and the concrete fix applied, since understanding why an issue occurred was treated as equally important to resolving it.

S3 attachment CORS failure	Direct client upload failed with CORS preflight error. Fix: Added <code>aws_s3_bucket_cors_configuration</code> allowing <code>PUT</code> , <code>GET</code> , <code>HEAD</code> from web origin.
ECS missing attachment bucket variable	ECS task could not generate pre-signed upload URLs. Fix: Updated ECS Terraform task definition to pass <code>AWS_S3_ATTACHMENT_BUCKET</code> environment variable.
Un-parameterized JPQL query fallback	JPQL query failed to bind user ID in repository layer. Fix: Added explicit <code>@Param('userId')</code> annotations to query methods in <code>TicketRepository.java</code> .
Docker tag mismatch in ECS	ECS deployment pulled stale container image. Fix: Updated <code>deploy.yml</code> to tag and push <code>:v1</code> alongside <code>:latest</code> to match task definition 4.
Vite API base URL fallback	Frontend made API calls to <code>localhost</code> in production. Fix: Set fallback in <code>axiosInstance.js</code> to the live ALB DNS endpoint.
Problem	Root Cause & Technical Fix Applied

Lessons Carried Forward

Several of these issues share a common thread: configuration that is correct in isolation but incomplete once services are wired together — a CORS policy that was fine for same-origin testing but not for the deployed frontend origin, or a task definition that worked until a new environment variable was introduced. The team's response was to make these dependencies explicit in Terraform and in the pipeline itself, rather than relying on manual configuration, so that a future contributor reproducing the environment from scratch would not need to rediscover the same fixes.

20. Individual Contribution Mapping (60 Marks Allocation)

Formal contribution mapping for the two project team members. The evaluation marks (60 Marks Total) and operational responsibilities are divided **50/50 equally** (30 Marks each), focusing exclusively on Cloud Architecture, Infrastructure-as-Code, Security Governance, CI/CD DevOps, Database Administration, and Quality Assurance, without any software development work assigned to either team member.

Likitha Arumilli	Cloud Infrastructure & DevOps Lead	IaC Terraform, VPC Network Architecture, ECS Fargate Container Orchestration, CI/CD Pipeline Automation, CloudWatch Observability	infrastructure/terraform/.github/workflows/deploy.yml modules/vpc/ modules/ecs/ modules/alb/	• Architected and provisioned the Multi-AZ VPC network (public/private/database subnets, ALB, Internet & NAT Gateways). • Defined ECS Fargate serverless cluster, task definitions, and container health checks. • Authored the zero-downtime GitHub Actions CI/CD pipeline for automated IaC validation, ECR container builds, and ECS deployments. • Integrated AWS Secrets Manager environment injection with ECS execution roles. • Configured CloudWatch log group aggregation (<code>/ecs/tkt-kc-dev-backend</code>) and Spring Boot Actuator monitoring rules. • Executed and validated full <code>terraform apply</code> and <code>terraform destroy</code> environment lifecycles.	30 / 60
Sandeep Singh	Cloud Security, Database & Operations Lead	Cloud Storage & S3 Security, Database Infrastructure & Migrations, Security Governance & RBAC Enforcement, Cost Optimization, Quality Assurance & Testing Verification	modules/s3/ modules/rds/ modules/security_groups/backend/.../db/migration/CHECKLIST.md cost_report.md	• Designed and provisioned AWS S3 buckets for static hosting (CloudFront integration) and attachment uploads. • Configured S3 CORS rules, bucket public-read policies, and pre-signed upload security constraints. • Provisioned RDS MySQL 8.0 Multi-AZ database instances, DB subnet groups, and Flyway versioned	30 / 60

				schema migration pipelines (V1 , V2). • Enforced network-level defense-in-depth security group ingress rules across ALB, ECS, and RDS. • Executed JUnit 5 integration test verification, RBAC role-isolation tests, and CORS boundary checks. • Conducted AWS Cost Explorer expenditure analysis (~\$52.27/month) and published cost control recommendations.	
Team Member	Allocated Role	Evaluation Area	Module / Files Managed	Contribution Summary & Deliverables	Marks (Out of 60)

Contribution Division Summary

Across the two contribution areas, **Likitha Arumilli** and **Sandeep Singh** covered Cloud Architecture & Infrastructure as Code, Container Orchestration & CI/CD Pipelines, Network & Storage Security, Database Administration & Migrations, System Observability, and Quality Assurance — the essential cloud engineering disciplines required to take a proof of concept from architectural design through to a fully deployed, highly secure, cost-optimized, and repeatably provisioned system.

Appendix A: Technology Stack Summary

Frontend Framework	React 18	Compiled with Vite; deployed as a static bundle
Frontend Charts	Recharts	Interactive SLA and analytics visualizations
Backend Framework	Spring Boot 3	REST API, runs on Java 21 (<code>eclipse-temurin</code>)
Security	Spring Security 6	Stateless JWT (HMAC-SHA256), BCrypt password hashing
Database	MySQL 8.0.35	AWS RDS, Multi-AZ, <code>db.t3.micro</code>
Schema Migrations	Flyway	Versioned SQL migrations applied automatically at startup
Containerization	Docker (multi-stage)	<code>eclipse-temurin:21-jre-alpine</code> minimal runtime image
Container Orchestration	Amazon ECS Fargate	Serverless container execution, no EC2 to manage
Infrastructure as Code	Terraform (HCL)	Modularized under <code>infrastructure/terraform/</code>
CI/CD	GitHub Actions	Test, build, and deploy pipeline (<code>.github/workflows/deploy.yml</code>)
Object Storage	Amazon S3	Static website hosting and pre-signed attachment uploads
Secrets Management	AWS Secrets Manager	Database credentials and JWT signing keys
Observability	Amazon CloudWatch Logs	Centralized container log aggregation (<code>/ecs/tkt-kc-dev-backend</code>)
Testing	JUnit 5	83 backend classes verified in CI pipeline
Layer	Technology	Notes

Together, this stack was selected to keep every layer either fully managed by AWS or fully defined as code, so that the proof of concept could be reproduced, evaluated, and torn down repeatably with minimal manual intervention.

Appendix B: Glossary of Terms

- **ALB (Application Load Balancer):** An AWS-managed Layer 7 load balancer that distributes incoming HTTP/HTTPS traffic across backend targets.
- **CIDR block:** A notation (e.g., `10.0.0.0/16`) describing a range of IP addresses assigned to a network or subnet.
- **CORS (Cross-Origin Resource Sharing):** A browser security mechanism that restricts which origins may make requests to a given resource, configured here on the S3 attachments bucket.
- **ECR (Elastic Container Registry):** AWS's managed Docker image registry, used here to store versioned backend images.
- **ECS Fargate:** A serverless compute engine for containers that removes the need to provision or manage underlying EC2 instances.

- **Flyway:** A database migration tool that applies versioned SQL scripts automatically, keeping schema changes tracked alongside application code.
- **HMAC-SHA256:** A cryptographic hashing algorithm used to sign and verify JWTs, ensuring tokens cannot be forged or altered.
- **IaC (Infrastructure as Code):** Defining cloud infrastructure in declarative configuration files (Terraform) rather than provisioning resources manually.
- **JWT (JSON Web Token):** A compact, signed token used to represent an authenticated user's identity without requiring server-side session storage.
- **Multi-AZ:** A deployment pattern that replicates resources across two or more AWS Availability Zones for high availability.
- **Pre-signed URL:** A time-limited, cryptographically signed URL that grants temporary access to a specific S3 object without exposing AWS credentials.
- **RBAC (Role-Based Access Control):** An authorization model that grants permissions based on a user's assigned role rather than per-user rules.
- **VPC (Virtual Private Cloud):** An isolated, logically separated network within AWS in which resources such as ECS tasks and RDS instances are deployed.

Appendix C: Recommendations for Production Readiness

The current deployment is deliberately scoped as a proof of concept. The items below outline what the team would prioritize before treating this as a production system, based on the gaps observed during the evaluation period.

1. **HTTPS Termination:** The ALB currently listens on port 80; a production deployment would add an ACM SSL/TLS certificate and an HTTPS listener, redirecting all HTTP traffic to HTTPS.
2. **Auto Scaling:** The ECS service currently runs a fixed task count; adding an Application Auto Scaling policy tied to CPU/memory or request count would let the service handle variable load without manual intervention.
3. **CloudWatch Alarms & Alerting:** The Actuator health and metrics endpoints already exist (Section 13); the next step is wiring them into CloudWatch Alarms and SNS notifications so the team is notified proactively.
4. **CloudFront CDN Distribution:** Fronting the S3 static website with Amazon CloudFront would reduce frontend latency for geographically distributed users and enable HTTPS on the custom domain.
5. **Backup & Retention Policies:** Formalizing RDS automated backup retention (7-30 days) and S3 lifecycle rules for older attachments would round out the operational posture.
6. **Cost Confirmation:** The cost figures in Section 15 should be continuously audited with AWS Cost Explorer as traffic grows.

None of these items block the current proof of concept from being evaluated on its stated objectives; they represent the natural next iteration once the platform moves from evaluation toward a production rollout.